

OPTSKILLS: Learning Generalizable Optimization Skills from Problem Archetypes via Cluster-Based Distillation

Haochen Yang^{1*}, Ke Zhao^{1*}, Mengyuan Ma¹, Xingyu Lu³,
Xiangfeng Wang¹, Hong Qian^{1,2 †}

¹East China Normal University, Shanghai, China

²Shanghai Innovation Institute, Shanghai, China

³Ant Group, China

M0kou@outlook.com, {kezha, 10222140418}@stu.ecnu.edu.cn,

sing.lxy@antgroup.com, xfwang@sei.ecnu.edu.cn, hqian@cs.ecnu.edu.cn

Abstract

Leveraging Large Language Models (LLMs) to automatically formulate and solve optimization problems from natural language has emerged as an efficient paradigm for automated optimization. However, existing methods still exhibit limited generalization: they are sensitive to superficial narrative variations, reuse experience mainly at the case level, and struggle to adapt to shifted or emerging problem types. We propose OPTSKILLS, an archetype-centric skill learning and reasoning agent system for optimization modeling and solving. To improve robust generalization, our system clusters problems by their underlying archetypes rather than surface narratives. To improve in-distribution generalization, it explores diverse modeling paradigms and solver configurations within each cluster, then distills successful trajectories into reusable workflow-level skills. To improve out-of-distribution generalization, it refines existing skills or expands the skill library using newly obtained trajectories. Our system achieves a state-of-the-art micro-averaged accuracy of 68.27% on datasets encompassing diverse problem types and scenarios. In addition, on MIPLIB-NL, a highly challenging large-scale and high-dimensional benchmark, it achieves 26.91% accuracy, outperforming DeepSeek-V3.2-Thinking by 4.53%. After skill learning on Nano-CO, it reaches 72.79% on the OOD NLCO benchmark. Code and skills are available at <https://github.com/fujiwaranoM0kou/OptSkills>.

1 Introduction

The advancement of the industrial field brings the efficient solution of operational optimization problems into sharp focus across academia and industry. However, the inherent complexity of optimization

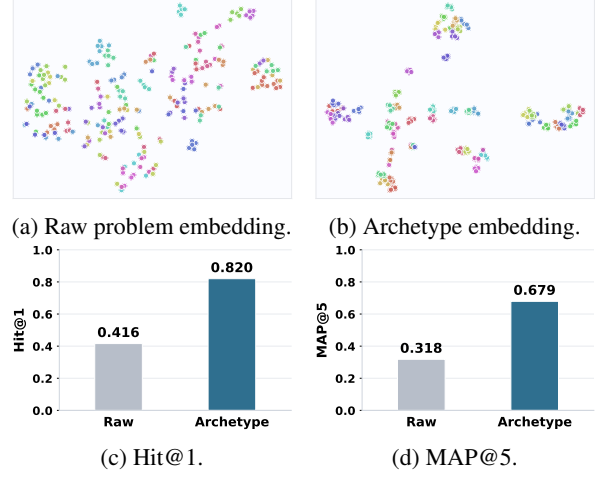


Figure 1: Embedding-space comparison on NANO-CO. Each point represents a problem instance, colored by manually verified problem-archetype labels. Top panels show t -SNE projections of raw problem-text embeddings and archetype embeddings; bottom panels report nearest-neighbor retrieval results in Hit@1 and MAP@5. Details are provided in Appendix F.1.

problems has necessitated reliance on domain experts to formulate mathematical models and develop solver code, a process that is both costly and expertise-intensive. Large language models (LLMs) have recently been explored as a promising tool for automating optimization modeling and solving. Existing studies have investigated multi-agent collaboration (AhmadiTeshnizi et al., 2024; Xiao et al., 2024), supervised fine-tuning (SFT) (Jiang et al., 2025a; Huang et al., 2025a), and reinforcement learning (Chen et al., 2026; Ding et al., 2026), achieving strong performance on standard benchmarks.

Recent studies have explored experience reuse for LLM-based optimization. AlphaOPT (Kong et al., 2026) constructs a self-evolving experience library that retrieves relevant experiences to guide similar problems, while LEAN-LLM-OPT (Liang et al., 2026) introduces a library of few-shot ex-

*These authors contributed equally to this work.

†Corresponding author

amples to help LLM agents construct structured workflows. However, these agents still suffer from limited generalization ability. In practical scenarios, this limitation appears in three forms. First, **robust generalization** requires the agent to identify the same optimization structure under different narratives, while avoiding sensitivity to superficial textual variations. Second, **in-distribution (ID) generalization** requires the agent to solve new instances of known optimization archetypes by reusing stable modeling and solving workflows. Third, **out-of-distribution (OOD) generalization** requires the agent to adapt when new domains introduce semantic variants of known archetypes or entirely new archetypes. These methods mainly organize reusable knowledge at the case level. Such case-level reuse can be sensitive to narrative changes, insufficient for distilling workflow-level knowledge shared by structurally equivalent problems, and limited in adapting to shifted or emerging problem types. This motivates an **archetype-centric view** of experience reuse, where optimization problems are organized by their canonical structures rather than surface textual similarity.

To provide empirical evidence for this view, this paper conducts exploratory experiments on the embedding-space clusterability of optimization problems under archetype-based representations. We use problems from the combinatorial optimization dataset NANO-CO, a self-constructed dataset described in the Appendix A. As shown in Figure 1b, when problems are represented by their archetype embeddings, instances with identical optimization structures form compact and well-separated groups under *t*-SNE (Van der Maaten and Hinton, 2008) projection. In contrast, Figure 1a shows that raw problem-text embeddings lead to lower intra-class compactness and severe inter-class overlap, indicating that surface narratives are not reliable indicators of reusable optimization workflows. Figures 1c and 1d further show that archetype embeddings improve nearest-neighbor retrieval in both Hit@1 and MAP@5. These results suggest that archetype-based representations provide a more suitable foundation for retrieving, clustering, and reusing optimization experience.

Based on the above observations, this paper propose OPTSKILLS, an agent system for archetype-centric workflow reuse in optimization modeling and solving. *To address the brittleness of case-level reuse under narrative variations*, OPTSKILLS clus-

ters optimization problems according to their underlying archetypes rather than surface-level textual similarity, allowing semantically different descriptions with the same canonical structure to share reusable experience. *To improve ID generalization within known optimization archetypes*, OPTSKILLS explores diverse modeling paradigms and solver configurations inside each archetype cluster, and distills intra-cluster trajectories into reusable workflow-level skills. *To enhance OOD generalization under distribution shifts*, OPTSKILLS further supports skill learning from newly obtained feasible trajectories, either refining existing skills for semantic variants of known archetypes or expanding the skill library for emerging problem types. In this way, OPTSKILLS contributes an archetype-centric experience reuse framework that directly targets robust generalization across narratives, ID generalization within known archetypes, and OOD adaptation to shifted optimization scenarios.

We evaluate OPTSKILLS on problem datasets encompassing multiple problem types and scenarios, where it achieves a state-of-the-art Micro-Avg. accuracy of 68.27%, outperforming the strongest baseline Trace2Skill by 4.81%. On the more challenging large-scale MIPLIB-NL benchmark, OPTSKILLS achieves 26.91% accuracy, outperforming DeepSeek-V3.2-Thinking by 4.53%. After skill learning on the self-constructed NANO-CO dataset, OPTSKILLS further reaches 72.79% accuracy on the OOD NLCO benchmark, showing the benefit of archetype-centric experience reuse under distribution shifts.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the proposed OPTSKILLS framework. Section 4 describes the experimental setup and analyzes the results. Section 5 concludes the paper.

2 Related Work

2.1 LLMs for Modeling and Solving Optimization Problems

Post-Training Methods. These approaches seek to encode optimization modeling directly into mathematical models through post-training of LLMs. Representative researches are based on multiple paradigms such as Supervised Fine-tuning (ORLM (Huang et al., 2025a), OptMATH (Lu et al., 2025), AutoOR (Motwani et al., 2026)), Preference Optimization (LLMOPT (Jiang et al., 2025a)), and Reinforcement Learning (SIRL (Chen et al., 2026)),

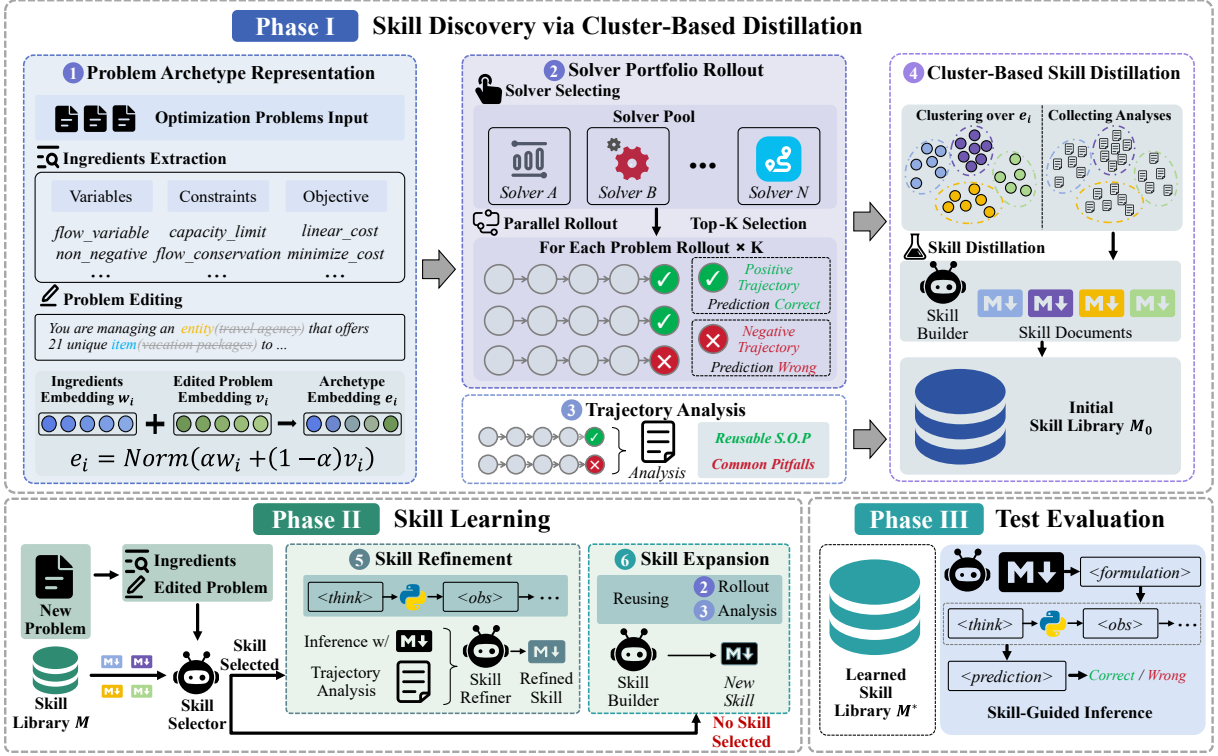


Figure 2: **An overview of OPTSKILLS.** OPTSKILLS consists of three phases. **Phase I:** (1) **Problem Archetype Representation**, (2) **Solver Portfolio Rollout**, (3) **Trajectory Analysis**, and (4) **Cluster-Based Skill Distillation** build the initial skill library M_0 . **Phase II:** for a new problem, skill selector routes it to (5) **Skill Refinement** or (6) **Skill Expansion**, yielding the learned skill library M^* . **Phase III:** the selected skill guides test-time inference.

OR-R1 (Ding et al., 2026)).

Agent-Based Methods. Rather than updating model parameters, these methods orchestrate multi-stage LLM workflows that decompose modeling, coding, solving, and verification into modular stages. Chain-of-Experts (Xiao et al., 2024) and OptiMUS (AhmadiTeshnizi et al., 2024) partition complex problems across specialized expert agents and organize them into structured agent topologies. ORMind (Wang et al., 2025) and ORThought (Yang et al., 2025b) incorporate cognitive reasoning and expert-level modeling heuristics to systematically map business requirements onto mathematical abstractions and solver implementations.

Experience-Enhanced Methods. Recent efforts begin integrating external experience, shifting the LLM-based optimization tasks from single-shot generation toward experience-augmented modeling. DRoC (Jiang et al., 2025b) and ConstraintLLM (Shi et al., 2025) decompose constraints into retrievable units to assist formal reasoning. MIRROR (Shi et al., 2026) leverages hierarchical search over exemplar libraries to provide task-specific modeling and code references. LEAN-LLM-OPT (Liang et al., 2026) retrieve ref-

erence examples from a curated reference dataset and construct structured modeling workflows to guide downstream formulation and code generation; AlphaOPT (Kong et al., 2026) build experience library that distill structured insights from failed attempts.

2.2 Skill Learning in LLM Agents

In contrast to task-specific experience augmentation, LLM agents equipped with skills offer a more general framework for cross-task knowledge accumulation and reuse. Skills preserve actionable task knowledge yet remain sufficiently abstract to support transfer across diverse problems. XSkill (Jiang et al., 2026a) separates operational experiences from structural skills. EvoSkill (Alzubi et al., 2026), Memento-Skills (Zhou et al., 2026), and AutoSkill (Yang et al., 2026) emphasize automatic skill discovery, routing, and evolution. Trace2Skill (Ni et al., 2026) inductively consolidates trajectory-local lessons into unified, conflict-free, and transferable skill catalogs. MemSkill (Zhang et al., 2026) recasts memory operations themselves as learnable skills, optimizing the skill library through a closed loop of controller,

executor, and designer.

3 Methodology

3.1 Preliminaries

Optimization Problem Archetype. An optimization problem archetype $a \in \mathcal{A}$ refers to a canonical optimization structure abstracted from concrete application scenarios. Given an archetype a , we denote its natural-language instance as $x = g_u(a)$, where u specifies the concrete scenario and g_u represents the verbalization process that expresses the underlying optimization structure as a natural-language problem statement. Ideally, x and its archetype a should induce the same optimization model, i.e., $f_{\text{model}}(a) = f_{\text{model}}(g_u(a))$. It indicates that optimization modeling should be invariant to surface scenario and narrative variations, problems that share the same archetype may differ substantially in scenario settings, narrative contexts, and natural-language surface forms, yet they typically share similar decision variables, objective functions, constraint patterns, and parameter configurations at the modeling level.

Skills. A skill $s \in \mathcal{S}$ is a reusable procedural knowledge document tailored to a class of optimization problem archetypes. Formally, $s = (m, w, p)$, where m denotes skill metadata, including its name and brief description; w denotes the modeling and solving workflow, covering decision variable, objective construction, constraint modeling, solver configuration, status checking, and result parsing; p denotes common pitfalls, which summarize frequent error patterns in modeling and solving. A skill example is provided in Appendix D.

3.2 An Overview of OPTSKILLS

We present OPTSKILLS, an agent system for natural-language optimization tasks. As shown in Figure 2, OPTSKILLS proceeds in three phases. **Phase I: Skill Discovery via Cluster-Based Distillation** constructs the initial skill library $\mathcal{M}_0$ from the dataset. It first derives archetype of problems through ingredients extraction and problem editing, then clusters problems in this representation space, and distills cluster-based skills from modeling and solving trajectories within each cluster. **Phase II: Skill Learning** refines and expands the library on newly encountered problems. **Phase III: Test Evaluation** retrieves skills relevant to the target problem from $\mathcal{M}$ and injects them into the agent as procedural guidance for inference.

3.3 Phase I: Skill Discovery via Cluster-Based Distillation

Problem Archetype Representation. Given a training problem x_i , the system first constructs an archetype embedding e_i associated with its archetype a_i . This embedding captures the problem’s modeling structure while reducing the influence of scenario-specific narratives. An extractor driven by LLM outputs optimization ingredients κ_i and an edited problem description $\tilde{x}_i$.

The ingredients κ_i include the variables, constraints, and objective function at an abstract level. They are not copied from the original text, but are designed to ignore concrete scenarios, entity names, and narrative backgrounds as much as possible. The edited text $\tilde{x}_i$ replaces scenario-specific narratives and entity names with generic placeholders while preserving the original parameters, constraints, and numerical information.

The archetype embeddings $e_i = \text{Norm}(\alpha \cdot w_i + (1 - \alpha) \cdot v_i)$ are calculated as the normalized weighted sum of w_i and v_i , which are the embeddings of κ_i and $\tilde{x}_i$, respectively. We employ Qwen3-Embedding-v3 as the embedding model, and the weight $\alpha \in [0, 1]$ controls the relative contributions of κ_i and $\tilde{x}_i$. Problems that share a similar archetype a are expected to have high similarity between their archetype embeddings e_i , which provides the basis for clustering.

Solver Portfolio Rollout. For each problem, this module explores multiple different modeling approaches and solver configurations, and collects solution trajectories. The rationale is that different modeling paradigms and solver configurations may vary in their applicability and stability for the same optimization problems. Given $\tilde{x}_i$ and κ_i , an LLM-based solver selector chooses a subset $\mathcal{B}_i$ consisting of the top- k most likely solvers from the candidate pool $\mathcal{B}$. For each solver $b \in \mathcal{B}_i$, the LLM first generates an optimization model z_i^b . It then iteratively generates executable solver code $c_{i,t}^b$ and receives execution observations $o_{i,t}^b$.

Assume that after T_i^b iterations of code generation and solving, a feasible solution $\hat{y}_i^b$ is obtained. We define the modeling and solving trajectory under the specific combination of modeling paradigm and solver configuration b as

$$\tau_i^b = (z_i^b, c_{i,1}^b, o_{i,1}^b, c_{i,2}^b, o_{i,2}^b, \dots, c_{i,T_i^b}^b, \hat{y}_i^b). \quad (1)$$

The trajectory set is $\mathcal{T}_i = \{\tau_i^b \mid b \in \mathcal{B}_i\}$. In the trajectory set, a trajectory is labeled positive if $\hat{y}_i^b$

matches the ground-truth answer y_i^* ; otherwise, it is labeled negative. This partitions $\mathcal{T}_i$ into positive trajectory set $\mathcal{T}_i^+$ and negative trajectory set $\mathcal{T}_i^-$.

Trajectory Analysis. Given $\mathcal{T}_i^+$ and $\mathcal{T}_i^-$, an LLM-based skill analyst is used to distill rollout trajectories into readable analyses. From positive trajectories, it extracts effective solver configurations and reusable modeling and solving procedures, summarized as standard operating procedures (SOPs). From negative trajectories, it identifies failure causes and summarizes them as common pitfalls:

$$q_i = A_\eta(\mathcal{T}_i^+, \mathcal{T}_i^-) = (r_i, p_i). \quad (2)$$

The skill analyst A_η returns two components: reusable SOPs r_i and the corresponding common pitfalls p_i . Each analysis q_i serves as an input to skill distillation, contributing an atomic unit of distilled knowledge.

Cluster-Based Skill Distillation. After obtaining e_i , $\mathcal{T}_i$, and analysis q_i for each training problem, problems are clustered according to their archetype embeddings, and reusable skills are distilled at the cluster level. Specifically, it applies DBSCAN (Ester et al., 1996) to embeddings, producing $\mathcal{C} = \text{DBSCAN}(\{e_i\}_{i=1}^N; \epsilon, m)$, where ϵ is the neighborhood radius and m is the minimum number of samples required to form a core point. Since e_i is normalized, cosine distance is used for clustering. Each cluster $C \in \mathcal{C}$ corresponds to a group of problems with a similar archetype.

For each cluster, an LLM-based skill builder aggregates the trajectory analysis of its member problems and distills a skill $s_C = B_\eta(\{q_i \mid i \in C\})$. The resulting set $\mathcal{M}_0 = \{s_C \mid C \in \mathcal{C}\}$ forms the initial skill library. Cluster-based distillation aggregates diverse positive and negative analyses under the same problem archetype, yielding procedural knowledge that is less dependent on instance-specific artifacts and more suitable for reuse.

3.4 Phase II: Skill Learning

After constructing the initial skill library $\mathcal{M}_0$, OPTSKILLS incrementally improves it on newly encountered problems. Given a new problem x , the system first applies the same ingredients extraction and problem editing steps as in Phase I. Based on these, an LLM-based skill selector identifies a candidate skill by comparing the problem against the names and descriptions of all skills in the current library. The selected candidate is then passed to an

LLM-based judge, which determines whether the skill should be accepted for solving the problem. If a matching skill is accepted, the system refines it using evidence from the new solution trajectory. Otherwise, the problem is treated as a potential new archetype and may be used to expand the library when sufficient evidence is available.

Skill Refinement. When an existing skill $s^* = (m^*, w^*, p^*)$ is selected, it is invoked as procedural guidance to solve the new problem x . The agent then generates a solution trajectory τ , which is evaluated according to the correctness of the answer. A positive trajectory provides evidence for reinforcing the reusable SOPs in w^* . A negative trajectory contributes failure evidence to the common pitfalls p^* . The trajectory is distilled into an analysis q by skill analyst A_η , and then used to update the selected skill s^* into a new skill s_{refined} by an LLM-based skill refiner.

The refiner updates the workflow component w^* using positive SOP evidence and augments the pitfall component p^* using negative evidence, while preserving the skill metadata and applicability boundary. The refined skill s_{refined} replaces s^* in the library.

Skill Expansion. When no suitable skill is found, the new problem is treated as a candidate for a new archetype. OPTSKILLS performs solver portfolio rollout to obtain a set of candidate trajectories $\mathcal{T}_x = \{\tau_x^b \mid b \in \mathcal{B}_x\}$. As in Phase I, the skill analyst summarizes them into trajectory analysis q .

To avoid adding unsupported procedural knowledge, OPTSKILLS expands the library only when at least one positive trajectory is available. In that case, a skill builder constructs a new skill from the trajectory analysis as $s_{\text{new}} = B_\eta(\{q\})$, and the library is updated to $\mathcal{M}_{t+1} = \mathcal{M}_t \cup \{s_{\text{new}}\}$.

3.5 Phase III: Test Evaluation

During testing, OPTSKILLS uses the learned skill library $\mathcal{M}^*$ as a fixed resource and does not perform any further updates. For each test problem x , the system first applies the same extractor as in previous phases to obtain the optimization ingredients κ and the edited problem $\tilde{x}$. An LLM-based skill selector then identifies the most relevant skill from $\mathcal{M}^*$ according to κ and $\tilde{x}$, relying on the skill names and descriptions as retrieval cues. The selected skill is used to guide the LLM agent during inference, and the prediction is finally evaluated against the ground-truth answer y^* .

Table 1: Comparison of the SA metric (Pass@1) across 5 benchmarks. OPTSKILLS-D and OPTSKILLS-Q indicate the OPTSKILLS based on DeepSeek-V3.2 and Qwen3-235B-A22b-instruct-2507, respectively. **Bold** indicates the 1st, underline indicates the 2nd, wavy underline indicates the 3rd.

Category	Models / Methods	Macro-Avg.	Micro-Avg.	Rank	OptiBench	Mamo.C	OptMATH	IndustryOR	ComplexOR
General Models	GPT-5.4	51.71	57.82	6	69.09	47.39	46.39	29.00	<u>66.67</u>
	Gemini-3.1-Pro	53.58	57.88	5	66.56	46.45	<u>54.22</u>	<u>34.00</u>	<u>66.67</u>
	Qwen3-235B	42.63	52.36	7	63.80	41.71	39.76	29.00	38.89
	DeepSeek-V3.2	43.21	49.45	9	62.64	27.49	40.36	30.00	55.56
Agent-Based Methods	Chain-of-Experts	38.84	52.36	8	62.31	51.66	35.54	28.00	16.67
	OptiMUS	34.14	45.18	13	60.99	27.49	22.89	26.00	33.33
	ORMind	36.69	47.37	11	61.82	34.60	21.69	32.00	33.33
	ORThought	41.92	46.91	12	57.69	38.86	26.51	31.00	55.56
Experience-Enhanced Methods	LEAN-LLM-OPT	41.23	49.27	10	62.15	43.13	22.89	28.0	50.0
	AlphaOPT	50.92	58.55	4	70.25	<u>54.50</u>	36.75	32.00	61.11
Skill-Based Methods	Trace2Skill	<u>56.97</u>	<u>63.46</u>	<u>2</u>	<u>75.21</u>	<u>54.03</u>	49.40	<u>34.00</u>	72.22
	OptSkills-Q	54.64	61.46	3	71.74	53.55	<u>54.22</u>	27.00	<u>66.67</u>
	OptSkills-D	62.04	68.27	1	77.02	63.51	61.45	36.00	72.22

4 Experiments

Datasets for Optimization Skills Construction.

To acquire problem instances encompassing diverse types and scenarios, we utilize the OptMATH-Train (Lu et al., 2025) dataset as a candidate pool. We then perform uniform sampling across problem types and random sampling across scenarios, yielding 300 samples for training OPTSKILLS. The distribution across problem types is as follows: Linear Programming (75), Mixed-Integer Linear Programming (75), Integer Programming (75), Non-Linear Programming (45), and Second-order Cone Programming (30), spanning 20 distinct problem scenarios.

OPTSKILLS first employs 150 instances for cluster-based skill distillation to derive an initial set of skills. Subsequently, OPTSKILLS continues to learn on the remaining 150 samples. This phase serves to both augment the skill repository with new skills and refine existing skills. Detailed experiment setups are provided in Appendix B.

Evaluation Benchmarks. For performance comparison, we adopt 5 benchmarks: OptiBench (Yang et al., 2025c), OptMATH-Bench (Lu et al., 2025), Mamo.C (Huang et al., 2025b), IndustryOR (Huang et al., 2025a), ComplexOR (Xiao et al., 2024). For Skill Learning evaluation, we employ the hard split of NLCO Benchmark (Jiang et al., 2026b). These benchmarks feature high problem difficulty and rich structural diversity. Detailed descriptions are provided in Appendix C.

Baselines. We benchmark our method against eleven distinct baselines in four categories. First, we include four general LLMs: GPT-5.4 (Ope-

nAI, 2026), Gemini-3.1-Pro (DeepMind, 2026), DeepSeek-V3.2 (DeepSeek-AI et al., 2025), and Qwen3-235B (Yang et al., 2025a); these models directly generate solving code via prompting in a non-tool-use manner. Second, we compare with four LLM-based agent frameworks: CoE (Xiao et al., 2024), OptiMUS (AhmadiTeshnizi et al., 2024), ORMind (Wang et al., 2025), and ORThought (Yang et al., 2025b). Third, we include two experience-enhanced agent-based approaches, AlphaOPT (Kong et al., 2026) and LEAN-LLM-OPT (Liang et al., 2026), both of which support knowledge accumulation and reuse. Finally, we include Trace2Skill (Ni et al., 2026) as a strong skill-based baseline, since it also leverages distilled skills to enhance agent performance. Detailed reproduction settings are provided in Appendix E.

Through extensive experiments, we aim to answer the following research questions:

Q1: How does the optimization modeling and solving performance of OPTSKILLS compare with baseline methods?

Q2: How do skills enhance the agent’s capability in modeling and solving optimization problems?

Q3: What advantages does the clustering strategy offer for skill utilization?

Q4: How well does OPTSKILLS generalize to in-distribution and out-of-distribution domains?

Q5: How effective is OPTSKILLS in handling real-world and large-scale optimization problems?

4.1 Main Results

We evaluate all methods using Pass@1 solving accuracy (SA), and summarize the results in Table 1.

Comparison of OPTSKILLS and Baseline Methods (To Q1). As summarized in Table 1, OPT-

Table 2: Ablation study of OPTSKILLS on the SA metric across 5 benchmarks. The table evaluates the contribution of each module by incrementally adding them to the baseline model. A checkmark indicates that the corresponding module is enabled, while an empty box indicates that it is disabled.

Experimental Groups	Tool-Call Agent	Skill Distillation	Cluster-Based Distillation	Skill Learning	OptiBench	Mamo.C	OptMATH	IndustryOR	ComplexOR
Group 1	☑	☐	☐	☐	73.22	53.08	54.21	34.00	66.67
Group 2	☑	☑	☐	☐	69.75	56.40	59.64	32.00	66.67
Group 3	☑	☑	☑	☐	74.71	59.24	60.84	35.00	72.22
Group 4	☑	☑	☑	☑	77.02	63.51	61.45	36.00	72.22

SKILLS (using DeepSeek-V3.2 as the LLM backbone) consistently achieves state-of-the-art performance across almost all benchmarks, attaining a micro-average pass@1 accuracy of 68.27% and demonstrating significant improvements over all baselines. To isolate the contribution of our system from the inherent capabilities of the underlying LLM backbone (DeepSeek-V3.2), we conduct an ablation study by using Qwen3-235B-A22b-instruct-2507 as the base model during inference. This configuration achieves a micro-average accuracy of 61.46%, comparable to that of Trace2Skill, and outperforms the majority of baselines. These results suggest that the performance gains stem from OPTSKILLS itself rather than from the choice of the backbone model.

Notably, OPTSKILLS exhibits pronounced advantages on challenging benchmarks such as OptMATH and Mamo.C, where it substantially outperforms prompt-based baselines. Even on the relatively simple OptiBench benchmark, OPTSKILLS maintains a significant accuracy margin.

4.2 Ablation Study

The Contribution of Skills to Agent Performance (To Q2). We conduct an empirical analysis of the cluster-based skill distillation process employed in Phase I. The Pass@1 accuracy of each condition across 5 benchmarks is reported in Table 2. Although augmenting the baseline with per-sample skills (Group 2) yields modest gains on challenging benchmarks such as Mamo.C and OptMATH, the overall improvement remains marginal. This stems from functional overlap among skills. Moreover, per-sample extraction induces instance-specific biases, and skill selection relies on descriptions that primarily indicate applicability conditions rather than generalizable patterns. Consequently, individual per-sample skills exhibit limited transferability across structurally similar problems.

Moreover, we find that the system yields 129 distinct skills, of which 81 (62.8%) are never in-

voked during inference in 5 benchmarks. However, these unselected skills are not inherently ineffective; when multiple semantically similar skills coexist, the model’s likelihood of identifying a successful skill for a specific instance degrades markedly, compromising solution reliability.

Enhancement of Skill Application via Clustering Strategies (To Q3). In contrast, the clustered skill library (Group 3) exhibits substantial gains over the baselines. This shows that clustering significantly enhances the overall quality of the skill library, thereby facilitating robust generalization across problems that share the same archetype.

In addition, we used cluster-based distillation to derive a set of 46 skills. Among them, only 7 skills remained unused throughout the inference process (15.2%). In contrast, the non-clustering baseline exhibited a higher proportion of skills never being invoked (62.8%). This indicates that the clustering-based distillation process effectively mitigates redundancy within the skill library, thereby enhancing the likelihood with which LLMs identify and retrieve the most appropriate skill for a given problem instance.

Since the clustering strategy has an impact on the quality of cluster-based skill generation, to validate the effectiveness of our proposed archetype-based clustering approach, we conduct a comparative analysis against the strategy that performs clustering directly on embeddings derived from raw problem text. Detailed results and discussions are provided in Appendix F.1.

Skill Learning on In-Distribution Data (To Q4-1). As summarized in Group 4 of Table 2, the proposed configuration demonstrates improvements in average accuracy across almost all benchmarks compared to Group 3. The exceptions are IndustryOR and ComplexOR, where the relatively high error rates (Xiao et al., 2025) may weaken the statistical significance of the observed gains. Additionally, ComplexOR contains a comparatively small number of samples, which further limits the

robustness of the comparison.

To investigate the growth dynamics of the skill library during skill learning and its impact on solving performance, we establish four checkpoints during OPTSKILLS’s learning process on OptiBench, corresponding to skill library sizes of 46, 49, 53, and 56. We independently evaluate the OPTSKILLS configured with each checkpoint’s skill library on the same benchmark. The solving accuracy on OptiBench exhibits a non-monotonic upward trend as the library grows from 46 to 56 skills (74.71% $\rightarrow$ 74.05% $\rightarrow$ 75.37% $\rightarrow$ 77.02%). Despite a minor dip at the intermediate checkpoint (49 skills), the overall trajectory demonstrates that expanding skill coverage within the same data distribution yields sustained performance gains, validating OPTSKILLS’s capacity for self-improvement through skill learning. The bar chart of the scaling phenomenon is shown in Appendix F.3.

4.3 Discussions

Skill Learning on Out-of-Distribution Data (To Q4-2). To assess OPTSKILLS’s generalization and adaptation ability under distribution shift, we first evaluate the initial skill library on the NLCO benchmark, an OOD combinatorial optimization benchmark. We then construct an OOD adaptation dataset, NANO-CO, via template-based synthesis. NANO-CO comprises combinatorial optimization problems whose problem-type distribution differs from the 300 training samples above. We use NANO-CO for skill learning and evaluate the adapted OPTSKILLS again on NLCO.

As reported in Table 3, OPTSKILLS-D with initial skill library (before learning) achieves a 5.96 percentage-point absolute improvement over Tool-Call Agent (68.56% vs. 62.60%), indicating that the initial skill library already exhibits nontrivial cross-domain transfer.

Based on the 46 skills acquired in the initial skill library, we expanded the skill library with 47 new skills on NANO-CO, resulting in a total of 93 skills. We then evaluated OPTSKILLS with the expanded skill library on the hard split of NLCO benchmark, achieving an accuracy of 72.79%, a 10.19 percentage-point absolute gain over the baseline. This suggests that OPTSKILLS can continuously expand its coverage of problem archetypes under shifted problem distributions.

Across all evaluated instances, successful trajectories exhibit a higher utilization rate of new skills compared to failed ones. Except for 9 failed sam-

Table 3: Performance comparison of OPTSKILLS on the NLCO dataset. Before Learning denotes direct evaluation using the initial skill library obtained by cluster-based distillation from 150 samples in the previous dataset. After Learning denotes the setting where OPTSKILLS further performs Skill Learning on our synthesized dataset.

Method	NLCO Accuracy (%)
Tool-Call Agent	62.60
Before Learning	68.56
After Learning	72.79

ples, 644 of the 1,565 successfully solved samples employ new skills. In contrast, only 176 of the 576 failed samples utilized new skills, with the remainder relying exclusively on existing skills. To elucidate the contribution of newly learned skills, we plot the usage rate of old and new skills as a pie chart, which is provided in Appendix F.4.

Solving Capability on High-Dimensional and Large-Scale Problems (To Q5). To evaluate the capability of OPTSKILLS on real-world and high-dimensional scenarios, we perform an evaluation on the challenging benchmark MIPLIB-NL (Li et al., 2026), which is characterized by large-scale decision variables and constraints. The results show that OPTSKILLS-D achieves a solving accuracy of 26.91% and exceeds DeepSeek-v3.2-thinking (22.38%) by 4.53 % reported in the paper of MIPLIB-NL. *Among the problems correctly solved by OPTSKILLS, the dimensionality of decision variables reaches up to 87,482.* This indicates that OPTSKILLS maintains competitive performance even when confronted with large-scale, industrial problem instances.

5 Conclusion

This paper proposes OPTSKILLS, an agent system that clusters optimization problems by problem archetypes and distills reusable skills from diverse solving trajectories. Across a diverse collection of problem spanning multiple problem types and scenarios, these archetype-level skills improve solving performance and generalization compared with strong baselines. Further experiments show that OPTSKILLS can adapt its skill library to new problem distributions and remains competitive on large-scale optimization benchmarks.

Limitations

OPTSKILLS relies on LLM-generated intermediate abstractions to construct problem archetype representations, including ingredients and edited problem. Although this design reduces the influence of surface narratives, errors in ingredient extraction or problem editing may propagate to clustering, skill selection, and downstream skill distillation. For example, missing a key constraint or incorrectly abstracting the objective may place a problem into an inappropriate cluster and lead to less reliable skill reuse. Future work should incorporate stronger verification mechanisms for archetype extraction, such as symbolic consistency checks. In addition, the quality of the learned skills library depends on the granularity of the clustering of archetypes. Over-fragmented clusters may produce redundant skills that increase the difficulty of skill selection, whereas over-merged clusters may combine different problem archetypes and yield ambiguous procedural guidance. This may partly explain why increasing the number of skills does not always lead to monotonic improvements across benchmarks. Although our sensitivity analysis suggests that the selected clustering setting provides a reasonable trade-off, developing adaptive clustering, skill merging, and skill pruning mechanisms remains an important direction for future work.

References

- Ali AhmadiTeshnizi, Wenzhi Gao, and Madeleine Udell. 2024. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In *Advances in Forty-first International Conference on Machine Learning*, Vienna, Austria.
- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. 2026. *EvoSkill: Automated skill discovery for multi-agent systems*. *arXiv preprint arXiv:2603.02766*.
- Yitian Chen, Jingfan Xia, Siyu Shao, Dongdong Ge, and Yinyu Ye. 2026. Solver-Informed RL: Grounding large language models for authentic optimization modeling. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Google DeepMind. 2026. *Gemini 3.1 pro*.
- DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, and 245 others. 2025. *DeepSeek-v3.2: Pushing the frontier of open large language models*. *Preprint*, arXiv:2512.02556.
- Zezen Ding, Zhen Tan, Jiheng Zhang, and Tianlong Chen. 2026. OR-R1: Automating modeling and solving of operations research optimization problem via test-time reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 228–236.
- Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, Portland, Oregon, USA, pages 226–231. AAAI Press.
- Chenyu Huang, Zhengyang Tang, Shixi Hu, Ruoqing Jiang, Xin Zheng, Dongdong Ge, Benyou Wang, and Zizhuo Wang. 2025a. ORLM: A customizable framework in training large models for automated optimization modeling. *Operations Research*.
- Xuhan Huang, Qingning Shen, Yan Hu, Anningzhe Gao, and Benyou Wang. 2025b. LLMs for mathematical modeling: Towards bridging the gap between natural and mathematical languages. In *Findings of the Association for Computational Linguistics 2025*, pages 2678–2710, Albuquerque, New Mexico.
- Caigao Jiang, Xiang Shu, Hong Qian, Xingyu Lu, Jun Zhou, Aimin Zhou, and Yang Yu. 2025a. LLMOPT: learning to define and solve general optimization problems from scratch. In *Advances in The Thirteenth International Conference on Learning Representations*, Singapore.
- Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. 2026a. *XSkill: Continual learning from experience and skills in multimodal agents*. *arXiv preprint arXiv:2603.12056*.
- Xia Jiang, Jing Chen, Cong Zhang, Jie Gao, Chengpeng Hu, Chenhao Zhang, Yaixin Wu, and Yingqian Zhang. 2026b. *Reasoning in a combinatorial and constrained world: Benchmarking llms on natural-language combinatorial optimization*. *Preprint*, arXiv:2602.02188.
- Xia Jiang, Yaixin Wu, Chenhao Zhang, and Yingqian Zhang. 2025b. DRoC: Elevating large language models for complex vehicle routing via decomposed retrieval of constraints. In *The Thirteenth International Conference on Learning Representations*.
- Minwei Kong, Ao Qu, Xiaotong Guo, Wenbin Ouyang, Chonghe Jiang, Han Zheng, Yining Ma, Dingyi Zhuang, Yuhang Tang, Junyi Li, Shenhao Wang, Haris Koutsopoulos, Hai Wang, Cathy Wu, and Jinhua Zhao. 2026. *AlphaOPT: Formulating optimization programs with self-improving llm experience library*. *Preprint*, arXiv:2510.18428.
- Zhong Li, Hongliang Lu, Tao Wei, Yuxuan Chen, Wenyu Liu, Yuan Lan, Fan Zhang, and Zaiwen Wen.

2026. Constructing industrial-scale optimization modeling benchmark. In Forty-third International Conference on Machine Learning.
- Kuo Liang, Yuhang Lu, Jianming Mao, Shuyi Sun, Chunwei Yang, Congcong Zeng, Xiao Jin, Hanzhang Qin, Ruihao Zhu, and Chung-Piaw Teo. 2026. [Large-scale optimization model auto-formulation: Harnessing llm flexibility via structured workflow](#). Preprint, arXiv:2601.09635.
- Hongliang Lu, Zhonglin Xie, Yaoyu Wu, Can Ren, Yuxuan Chen, and Zaiwen Wen. 2025. Opt-MATH: A scalable bidirectional data synthesis framework for optimization modeling. In Forty-second International Conference on Machine Learning.
- Sumeet Ramesh Motwani, Chuan Du, Aleksander Petrov, Christopher Davis, Philip Torr, Antonio Papania-Davis, and Weishi Yan. 2026. [AutoOR: Scalably post-training llms to autoformalize operations research problems](#). arXiv preprint arXiv:2604.16804.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjin Jiang. 2026. [Trace2Skill: Distill trajectory-local lessons into transferable agent skills](#). arXiv preprint arXiv:2603.25158.
- OpenAI. 2026. [Introducing gpt-5.4](#).
- Weichun Shi, Minghao Liu, Wanting Zhang, Langchen Shi, Fuqi Jia, Feifei Ma, and Jian Zhang. 2025. [ConstraintLLM: A neuro-symbolic framework for industrial-level constraint programming](#). In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 15999–16019, Suzhou, China. Association for Computational Linguistics.
- Yifan Shi, Jialong Shi, Jiayi Wang, Ye Fan, and Jianyong Sun. 2026. [MIRROR: A multi-agent framework with iterative adaptive revision and hierarchical retrieval for optimization modeling in operations research](#). arXiv preprint arXiv:2602.03318.
- Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-sne. Journal of machine learning research, 9(11).
- Zhiyuan Wang, Bokui Chen, Yinya Huang, Qingxing Cao, Ming He, Jianping Fan, and Xiaodan Liang. 2025. [ORMind: A cognitive-inspired end-to-end reasoning framework for operations research](#). In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track), pages 104–131, Vienna, Austria. Association for Computational Linguistics.
- Ziyang Xiao, Jingrong Xie, Lilin Xu, Shisi Guan, Jingyan Zhu, Xiongwei Han, Xiaojin Fu, WingYin Yu, Han Wu, Wei Shi, Qingcan Kang, Jiahui Duan, Tao Zhong, Mingxuan Yuan, Jia Zeng, Yuan Wang, Gang Chen, and Dongxiang Zhang. 2025. [A survey of optimization modeling meets llms: Progress and future directions](#). In Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI '25.
- Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Jessica Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, and Gang Chen. 2024. Chain-of-Experts: When LLMs meet complex operations research problems. In The Twelfth International Conference on Learning Representations, Vienna, Austria.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025a. [Qwen3 technical report](#). Preprint, arXiv:2505.09388.
- Beinuo Yang, Qishen Zhou, Junyi Li, Chenxing Su, Panagiotis Angeloudis, and Simon Hu. 2025b. [ORTThought: Benchmarking and automating logistics optimization modeling](#). arXiv preprint arXiv:2508.14410.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. 2026. [AutoSkill: Experience-driven lifelong learning via skill self-evolution](#). arXiv preprint arXiv:2603.01145.
- Zhicheng Yang, Yiwei Wang, Yinya Huang, Zhijiang Guo, Wei Shi, Xiongwei Han, Liang Feng, Linqi Song, Xiaodan Liang, and Jing Tang. 2025c. OptiBench meets ReSocratic: Measure and improve LLMs for optimization modeling. In The Thirteenth International Conference on Learning Representations, Singapore.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. [MemSkill: Learning and evolving memory skills for self-evolving agents](#). arXiv preprint arXiv:2602.02474.
- Huichi Zhou, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, Menglong Zhang, Yihang Chen, Jinsong Li, Runyu Yang, Qiangbin Liu, Xinlei Yu, Jianmin Zhou, Na Wang, Chunyang Sun, and Jun Wang. 2026. [MementoSkills: Let agents design agents](#). arXiv preprint arXiv:2603.18743.

Appendix

A Data Synthesis

To evaluate and enhance the generalizable modeling capability of OPTSKILLS on combinatorial optimization problems, we constructed a collection of synthetic combinatorial optimization instances, named NANO-CO, as an adaptation set for skill learning before evaluation on the held-out NLCO

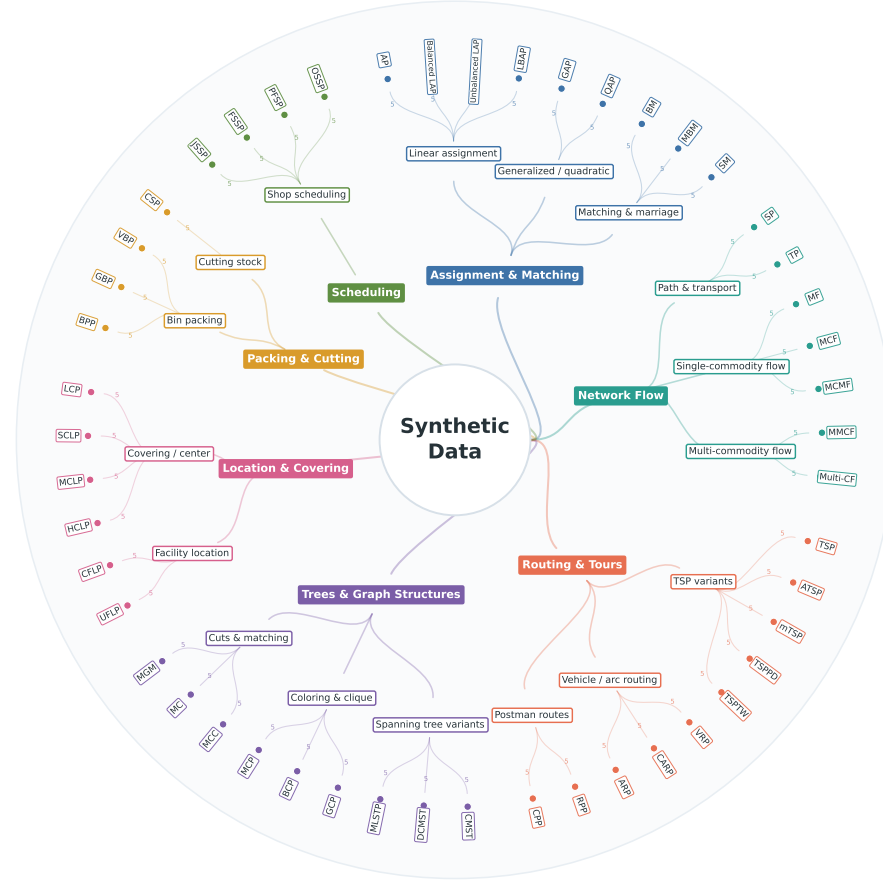


Figure 3: **Taxonomy of the synthetic data.** The dataset contains 49 combinatorial optimization problem types grouped into seven families. Each leaf node represents one problem type.

benchmark. Figure 3 summarizes the taxonomy of this synthetic dataset, including its high-level problem families, subfamilies, and individual problem types.

We manually curated a set of common combinatorial optimization problems and their representative variants. Based on these problem categories, we further developed 49 seed code templates, with each seed template corresponding to one problem type.

All seed templates were manually inspected and verified using solvers. Particular attention was paid to potential implementation issues in the objective functions, constraints, decision-variable definitions, and parameter settings that could affect optimality, ensuring that each template can consistently generate instances with verifiable global optimal solutions. Then, while preserving the original mathematical structure, we used large language models to generate five new themes with realistic business contexts for each template, such as logistics scheduling, emergency department rostering,

and semiconductor manufacturing.

To ensure that thematic rewriting does not alter the original combinatorial optimization structure, we established a strict entity-mapping relationship for each new theme. For example, a “machine” in the original problem may be mapped to an “operating room,” while its modeling role, constraint relationships, and parameter semantics remain unchanged. For each themed instance, we further injected an independent random seed to generate the corresponding numerical parameters and invoked solvers to verify both feasibility and global optimality. This process yielded a total of 245 training instances.

To contextualize the relationship between NANO-CO and NLCO, we manually compared their problem categories. An NLCO task category is counted as covered if it has an exact or close counterpart in NANO-CO. As shown in Figure 4, NANO-CO covers 22 of the 43 NLCO task categories, indicating substantial overlap while still leaving room for NLCO-specific problem types.

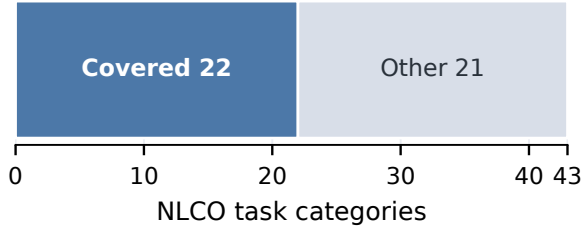


Figure 4: **NLCO task-category coverage** by NANO-CO exact or close counterparts.

Regarding instance scale, the number of constraints in NANO-CO ranges from 1 to 5, with an average of 2.8, and the number of decision variables ranges from 9 to 310, with an average of 48.

B Experiment Setup

B.1 Configuration of OPTSKILLS

Throughout all experiments, DeepSeek-V3.2 (non-thinking mode) serves as the uniform underlying LLM for all modules of OPTSKILLS. The Embedding model is Qwen3-Embedding-v3. We set the maximum agent-loop turns to 12 and set the temperature of the LLM backend to 0 across all stages. When fusing the archetype embeddings, we set the parameter α to 0.55. In the solver portfolio stage, the top- k parameter is set to 3. For DBSCAN clustering, the density parameter ϵ is set to 0.05, and the minimum number of samples m is set to 1. The random seed for training-data shuffling is set to 42.

B.2 Configuration of Baselines

For a fair comparison, all agent-based baselines are configured consistently with OPTSKILLS. Specifically, DeepSeek-V3.2 in non-thinking mode is used as the underlying LLM. Unless otherwise specified, the remaining implementation details follow the default settings of each baseline.

For AlphaOPT and Trace2Skill, we make necessary adaptations to ensure compatibility with our experimental setting and evaluation protocol. The detailed modifications are provided in Appendix E.6 and Appendix E.7.

C Benchmarks

To ensure the reliability and consistency of our experimental results, we adopt the cleaned and corrected versions of OptiBench, Mamo.C, OptMATH, IndustryOR, and ComplexOR from LLMOPT (Jiang et al., 2025a), rather than the original datasets.

C.1 OptiBench

OptiBench is a large-scale benchmark designed to evaluate the end-to-end capability of large language models in solving optimization problems from complex inputs. It covers a diverse set of optimization settings, including linear programming, nonlinear optimization, and problems involving tabular data, reflecting practical scenarios where optimization tasks may be specified in heterogeneous formats. By incorporating nonlinear objectives or constraints as well as table-based problem information, OptiBench provides a comprehensive testbed for assessing whether models can accurately understand problem descriptions, formulate optimization models, and produce correct solutions across multiple forms of input.

C.2 Mamo.C

Mamo.C refers to the Complex_LP subset of the Mamo benchmark, which is designed to evaluate the mathematical modeling ability of large language models through solver-based answer verification. It contains 211 undergraduate-level optimization problems integrating LP and MILP structures. In this benchmark, a model is required to translate a natural-language problem description into a solver-readable optimization formulation, and the generated formulation is evaluated by comparing the solver-produced optimal value with the ground-truth answer. Since Mamo.C involves more complex decision variables and constraints than the Easy_LP subset, it provides a stricter testbed for evaluating optimization modeling capability.

C.3 OptMATH

OptMATH-Bench is a difficult optimization modeling benchmark derived from the OptMATH bidirectional data synthesis framework. It consists of hard instances selected through solver-based validation and rejection sampling, with problem descriptions that are substantially longer than those in NL4OPT and MAMO. The benchmark covers a broad range of optimization problem types, including LP, MILP, IP, NLP, and SOCP, providing a challenging testbed for evaluating whether large language models can formulate and solve complex long-context optimization problems.

C.4 IndustryOR

IndustryOR is an industrial benchmark introduced in the ORLM framework for evaluating large language models on practical operations research prob-

lems. Unlike benchmarks that mainly focus on textbook-style or homogeneous linear programming instances, IndustryOR is designed to reflect real-world optimization modeling scenarios across multiple industries. It contains 100 practical OR problems covering five types of optimization tasks, including linear programming, integer programming, mixed-integer programming, nonlinear programming, and other optimization problems, with instances organized into three difficulty levels. The benchmark evaluates models by execution accuracy, where a generated model is considered correct if the optimal value obtained by executing the solver code matches the ground-truth optimal value.

C.5 ComplexOR

ComplexOR is a benchmark introduced by Chain-of-Experts (CoE) for evaluating LLMs on complex operations research modeling and programming problems. Unlike elementary NL4Opt-style instances, ComplexOR focuses on more realistic and challenging OR problems whose descriptions may involve implicit constraints, domain-specific terminology, and long reasoning chains. Models are required not only to understand the natural-language problem statement, but also to construct an appropriate optimization model and generate executable solver code. The benchmark evaluates whether the generated program can pass test cases annotated by OR specialists, thereby assessing the model’s end-to-end capability in complex OR problem formalization and solving.

C.6 NLCO

NLCO was originally proposed to evaluate the direct reasoning ability of large language models in natural-language combinatorial optimization. In its original protocol, given a natural-language decision-making scenario, the model directly outputs a discrete solution without writing code or invoking external solvers, and the output is evaluated for feasibility and optimality against problem constraints, objective functions, and solver-annotated reference solutions. Covering 43 classes of combinatorial optimization problems, NLCO spans a broad range of structures, including routing, scheduling, packing, assignment, facility location, and graph optimization.

In this paper, we use NLCO differently from its original tool-free evaluation setting. Rather than testing direct discrete-solution generation, we

adopt NLCO as an out-of-distribution benchmark with natural-language problem statements, diverse combinatorial optimization structures, and verifiable reference answers. This enables us to examine whether OPTSKILLS can generalize beyond the LP-, MILP-, and NLP-centered benchmarks used in our main experiments to a wider spectrum of combinatorial optimization archetypes.

Specifically, we evaluate OPTSKILLS on NLCO’s most difficult Set-L, which contains 43 problem types with 50 instances each, totaling 2,150 test samples. None of these instances is used during OPTSKILLS’ skill learning or skill evolution process. During evaluation, OPTSKILLS takes the original NLCO problem statement as input and obtains the final answer through skill retrieval, mathematical modeling, code generation, and execution. The output is then assessed using the ground truth provided by NLCO. Since Set-L enlarges the combinatorial search space and strengthens global constraint dependencies, it provides a rigorous test of OPTSKILLS’ generalization to unseen combinatorial optimization structures.

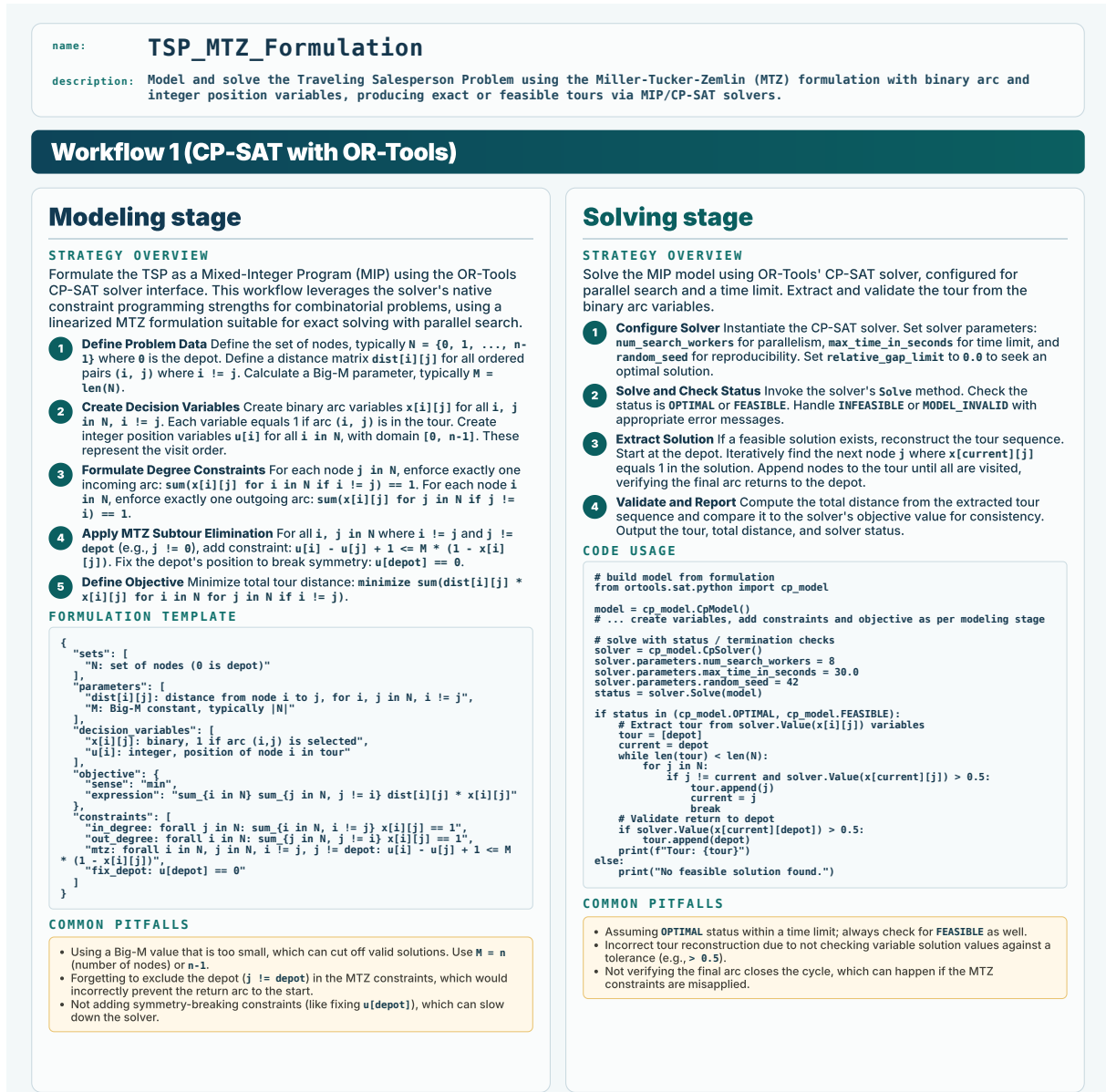
C.7 MIPLIB-NL

MIPLIB-NL is a high-dimensional and industrial-scale optimization modeling benchmark designed to expose the limitations of existing LLM-based optimization systems on large and difficult problems. It is constructed from real MILP instances in MIPLIB 2017 through a structure-aware reverse construction pipeline, yielding 223 verified natural-language-to-optimization instances with one-to-one correspondence to the original mathematical formulations. Since MIPLIB-NL targets problems whose scale can reach 10^3 – 10^6 variables and constraints, it provides a challenging testbed that is substantially closer to real industrial optimization than toy-scale benchmarks.

In this work, we adopt MIPLIB-NL to assess the capability of OPTSKILLS in handling high-dimensional optimization problems, especially whether the retrieved skills can still support reliable modeling and solving under large-scale, structurally complex settings.

D Skill Example

An example of a skill is provided in Figure 5. A skill contains multiple workflows, we show only one representative workflow.



nisms.

E.3 Reproduction of ORMind

ORMind is a cognitive-inspired end-to-end reasoning framework for operations research problems. It follows a structured workflow that transforms natural-language requirements into mathematical models and executable solver code through several modules, including a Semantic Encoder, Formalization Thinking, an Executive Compiler, a System 2 Reasoner, and a Metacognitive Supervisor. A key feature of ORMind is its use of counterfactual reasoning to identify potential formulation or implementation errors and refine the generated solution. In our reproduction, we use the default setting of the original implementation, including its module configuration, memory-pool-based workflow, counterfactual reasoning mechanism, and syntax-error analysis.

E.4 Reproduction of ORThought

ORThought is a structured dual-agent framework for automated optimization modeling. It decomposes the solving process into two core agents: a Model Agent that performs problem understanding, mathematical modeling, and Gurobipy code generation, and a Solve Agent that executes the generated code and refines it through a Detection–Diagnosis–Repair workflow. In our reproduction, we follow the default setting of the original implementation, including its dual-agent architecture, prompt templates, structured reasoning process, and execution-based repair mechanism. No additional task-specific tuning is performed, and ORThought is evaluated under the same problem inputs and evaluation protocol as other baselines.

E.5 Reproduction of LEAN-LLM-OPT

LEAN-LLM-OPT is a lightweight agentic workflow-construction framework for LLM-assisted optimization model auto-formulation. It uses upstream agents to identify the problem type and construct a structured workflow from reference examples, and a downstream model-generation agent follows this workflow to generate the final optimization formulation and executable solver code. The original framework relies on a reference dataset, Ref-Data, to support problem classification and workflow construction, and further incorporates CSV-based data-retrieval tools for large-scale instances with external datasets.

In our reproduction, we enable its reference-based workflow construction using Ref-Data and use the WITHOUT CSV setting, since our evaluation inputs do not rely on external CSV files. Apart from this adaptation, we keep its default workflow-construction and model-generation procedure.

E.6 Reproduction of AlphaOPT

AlphaOPT is an experience-learning framework for natural-language optimization modeling. Instead of updating model parameters, it constructs a self-improving experience library of solver-verified modeling insights. Each insight is represented with explicit applicability information, including its taxonomy, condition, explanation, and example, so that it can be retrieved and reused for later optimization tasks. AlphaOPT operates through two stages: Library Learning extracts structured insights from failed attempts and organizes them in a hierarchical taxonomy, while Library Evolution refines the applicability conditions of stored insights using aggregate evidence across tasks to reduce over- and under-generalization.

In its original experimental setup, AlphaOPT performs a train-test split on the benchmark. To ensure a fair comparison in our reproduction of AlphaOPT, we use the same training set as our method and expand its experience library from scratch.

E.7 Reproduction of Trace2Skill

Trace2Skill is an automatic skill construction and adaptation framework for LLM agents. It distills reusable skills from execution trajectories by first collecting a pool of agent traces, then using parallel success and error analysts to propose trajectory-level skill patches, and finally consolidating these patches into a unified, conflict-free skill directory through hierarchical merging. The resulting skill is used as a declarative artifact without parameter updates or test-time retrieval.

For Trace2Skill, we modified part of its prompts to adapt the method to the optimization problem setting. Specifically, we set the maximum agent-loop turns to 12 and used the same training set as our method.

F Additional Experiments

F.1 KNN-Based Validation of Embedding-Space Clusterability

To further validate the discriminative capability of clustered skills and the validity of using archetypes

Table 4: KNN retrieval performance comparison between raw problem and archetype representations across multiple ranking metrics.

Experiment Group	Hit@1	Hit@3	Hit@5	Precision@5	Recall@5	MRR	MAP@5
Raw Problem	0.416	0.690	0.8080	0.344	0.430	0.582	0.318
Archetype	0.820	0.939	0.971	0.596	0.745	0.885	0.679

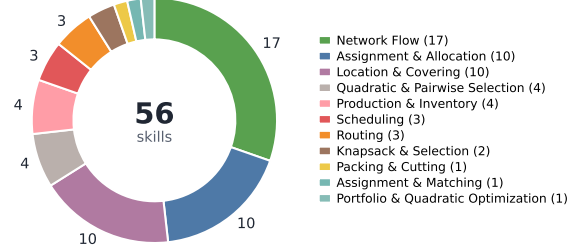
as the fundamental unit of abstraction, we conduct a K-Nearest Neighbor (KNN) validation. We establish a baseline condition in which raw problem texts are directly embedded to obtain representations $\tilde{e}$, and we compare these against our proposed archetype embeddings e . This experiment assesses the extent to which raw-text embeddings exhibit clusterable structure according to structural similarity rather than superficial semantic similarity in the embedding space.

We compute the following five metrics: **(1) Hit Rate (HR@k)**: Whether the top- k neighbors contain at least one sample from the same archetype group. **(2) Precision@k**: The proportion of same-archetype samples among the top- k neighbors. **(3) Recall@k**: The proportion of same-archetype samples retrieved among all members of that archetype group. **(4) Mean Average Precision (MAP)**: The mean of average precision scores across all queries. **(5) Mean Reciprocal Rank (MRR)**: The mean reciprocal rank of the first same-archetype neighbor.

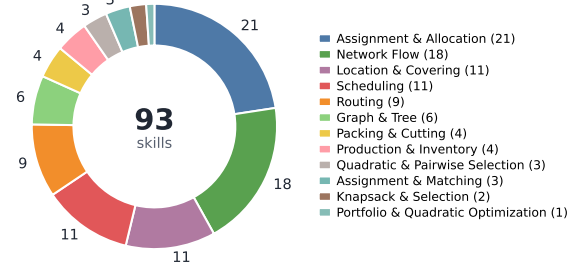
All metrics range from 0 to 1, with higher values indicating stronger structural consistency and better clusterability in the embedding space. As reported in Table 4, raw-text embeddings ($\tilde{e}$) perform significantly worse than archetype embeddings (e), failing to place problems that share the same modeling and solving patterns close to each other. Conversely, archetype-based embeddings achieve substantially higher scores across all KNN metrics, indicating that skills distilled via archetype clustering exhibit clear type-level discriminability. This corroborates the validity of using archetypes as the core abstraction unit for problem representation and skill organization.

F.2 Analysis of Skill Coverage

Figure 6 visualizes the archetype-level coverage of the learned skill library. The skill library learned from OPTMATH-TRAIN contains 56 skills and already spans 11 optimization categories, including Network Flow, Assignment & Allocation, Location & Covering, Quadratic & Pairwise Selection, Production & Inventory, Scheduling, Routing, Knapsack & Selection, and Packing & Cutting.



(a) OPTMATH-TRAIN.



(b) NANO-CO.

Figure 6: **Skill coverage on problem family.** Top: coverage of the skill library learned from OPTMATH-TRAIN. Bottom: coverage after learning on NANO-CO.

sack & Selection, and Packing & Cutting. This suggests that the initial library captures a broad set of reusable modeling and solving patterns rather than being dominated by a single problem family.

Before evaluating on the OOD NLCO benchmark, we perform continued skill learning on our synthesized NANO-CO data. After this stage, the skill library expands to 93 skills and shows stronger coverage of combinatorial optimization structures, especially Assignment & Allocation, Network Flow, Location & Covering, Scheduling, Routing, Graph & Tree, and Packing & Cutting. These categories are highly aligned with the structural characteristics of NLCO.

This analysis confirms that OPTSKILLS has strong Skill coverage: the initial library provides broad in-domain archetype coverage, while continued learning on NANO-CO further extends the library toward OOD combinatorial structures.

Since no skills are learned from NLCO itself, the expanded coverage supports the ability of OPT-SKILLS to transfer newly acquired skills to unseen problem distributions.

F.3 The Scaling Exploration of Skills on OptiBench

Taking OptiBench as an example, we observe a clear scaling law in solving accuracy as the number of skills expands during training, and the bar chart of this scaling phenomenon is shown in Fig 7.

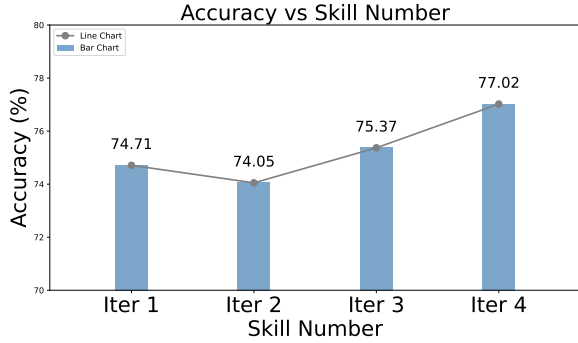


Figure 7: The solving accuracy of OPTSKILLS (based on DeepSeek-V3.2) on OptiBench with different numbers of skills in the skill library.

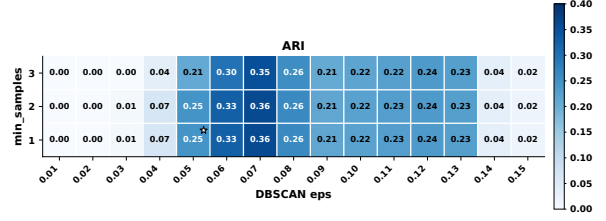
F.4 The Usage Statistics of New and Old Skills on NLCO Benchmark

On the NANO-CO dataset, OPTSKILLS expands the initial set of 46 existing skills by acquiring 47 new ones. Figure 9 illustrates the utilization patterns of these old and new skills for both successfully solved and failed instances on the NLCO benchmark. The results demonstrate that the newly acquired skills contribute substantially to performance on this benchmark.

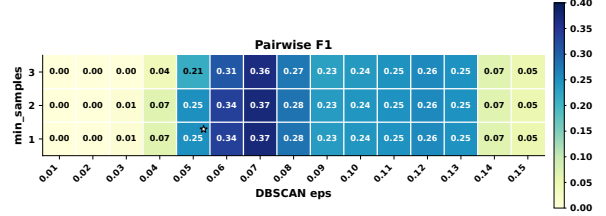
F.5 DBSCAN Sensitivity Analysis

We further evaluate the sensitivity of the archetype construction step to DBSCAN hyperparameters on NANO-CO. We fix the archetype embedding used in the main experiments and vary the DBSCAN radius ϵ from 0.01 to 0.15, together with $\text{min_samples} \in \{1, 2, 3\}$. The ground-truth problem types are used only for post-hoc diagnostics and are not used during clustering.

Figure 8 reports ARI and Pairwise F1 under different hyperparameter settings. Very small ϵ values lead to over-fragmented clusters and near-zero agreement with the problem-type labels. Performance improves in the moderate range of ϵ , with

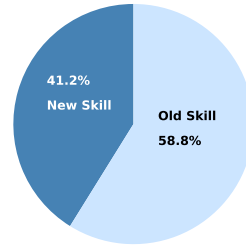


(a) ARI

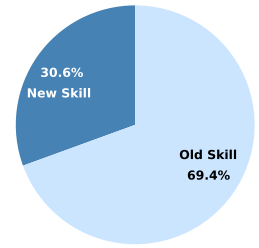


(b) Pairwise F1

Figure 8: DBSCAN sensitivity analysis on NANO-CO archetype embeddings. The star marks the setting used in our main experiments.



(a) The skills usage of the successful samples.



(b) The skills usage of the failed samples.

Figure 9: The utilization of newly acquired and existing skills across the subsets of correctly solved and failed instances within the NLCO benchmark.

the best diagnostic scores around $\epsilon = 0.07$. Although $\epsilon = 0.07$ gives the best post-hoc diagnostic score, we use $\epsilon = 0.05$ as a more conservative label-free setting to avoid over-merging structurally different archetypes.

G Case Study

To make the effect of skill injection more concrete, we include a representative case study in which the task is a cardinality-constrained pairwise selection problem with directed and asymmetric pairwise benefits. We compare a skill-guided trajectory with a no-skill trajectory on the same instance. Figure 10 summarizes the two trajectories and highlights where the semantic error occurs.

Skill-guided trajectory Correct 127	No-skill trajectory Incorrect 65
INJECTED KNOWLEDGE	
Injected skill fragment. • Class: Cardinality-Constrained Pairwise Selection. • Set: $P = \{(i,j): i \neq j\}$, ordered pairs. • Weight: $\text{benefit}(i,j)$ is directed/asymmetric. • Activation: $y(i,j) = x[i] * x[j]$. • Objective: maximize $\sum \text{benefit}(i,j) y(i,j)$.	No task-specific skill. Only generic modeling guidance is available. The agent receives no explicit warning that the 20-entry benefit table should remain directed/asymmetric.
REASONING TRACE	
1. Preserve directed pair set $P = \{(i,j) \text{ for } i \text{ in } N \text{ for } j \text{ in } N \text{ if } i \neq j\}$ maximize $\sum (\text{benefit}(i,j)) * y(i,j) \text{ for } (i,j) \text{ in } P$ Total benefit: 127	1. Correct solution, then self-doubt Selected nodes: [1, 2, 3] Objective value: 127 The agent treats this as double-counting: Total benefit ... 63.5 counts each unordered pair twice
2. Verify against the same objective Selected nodes: [1, 2, 3] Total benefit: 127 (1,2)=24, (2,1)=22 (1,3)=17, (3,1)=21 (2,3)=18, (3,2)=25 Final answer: 127	2. Optimizes a different problem $\text{benefit} = \{(0,1):16, \dots, (3,4):17\}$ # only define for $i < j$ Selected nodes: [0, 2, 4] Objective value: 65 Final answer: 65
INSIGHT	
Skill effect. The skill stabilizes the semantic choice $P = \{(i,j): i \neq j\}$. The correct set [1,2,3] yields $24+22+17+21+18+25=127$.	Failure mode. The no-skill run finds 127 but changes the task to unordered pairs $i < j$. The error is modeling semantics, not solver capability.

Figure 10: **Case study of skill-guided optimization modeling.** A comparison of the skill-guided and no-skill trajectories on the same optimization instance.

H Prompts

We provide the detailed prompts below.

H.1 Prompt of LLM baseline

PROMPT

You are a practical function-calling optimization research agent. Follow the 3-stage guidance and rules below to complete the task:

- ### Modeling and solving the problem step by step
1. ****Modeling Stage**:** First output a valid formulation in JSON format wrapped in `<formulation>` tags.
 2. ****Solving Stage**:** Generate the solver-based Python code and wrapped in `<solver_code>` tags.
 - you may use the python-based modeling language as follows: ortools, pyomo.
 - you may use the solver as follows: gurobi, clp, glpk, ipopt, highs, cbc, scip, mindtpy. You need to select an appropriate solver based on the type of the problem (linear or nonlinear, integer or continuous, and so on).
 - Solver code must print `f"RESULT: {{objective_value}}"`.

Problem Description
`<problem_description>`
`{problem_description}`
`</problem_description>`

Output the result as follows:
`<formulation>`
`{formulation}`
`</formulation>`

```
<solver_code>
{solver_code}
</solver_code>
```

H.2 Extractor Prompts

EXTRACTOR_PROMPT_EVAL

You are an optimization-structure extractor.

Your goal is to ignore surface semantics and recover the underlying optimization structure.

Different stories may describe the same mathematical problem class. You must normalize them into the same structural representation whenever the variables, constraints, and objective are essentially the same.

- You must do two things on the same problem:
- 1) extract scenario-agnostic optimization keywords into three slots;
 - 2) minimally edit the problem text into a scenario-neutral version.

Task A: Keyword extraction

- Extract high-signal structural keywords and place them into:
 - ``variable``
 - ``constraint``
 - ``objective``
- The goal is not only mathematical correctness, but also discriminative power: the extracted keywords should help distinguish different canonical optimization classes even when their algebraic forms are similar.
- The extracted keywords should capture the structural signature of the problem: the decision roles of variables, the coupling mechanisms in constraints, and the operational intent of the objective.
- Prefer canonical optimization roles, mechanisms, and structural motifs over generic algebraic descriptions.
- Prefer structurally discriminative keywords that best characterize the problem class, rather than the most generic mathematical-form label.
- Preserve distinctions between problem classes even when they share similar variable types, linear constraints, or objective form.
- If a keyword is needed, use lowercase snake_case and keep it scenario-agnostic.
- Use generic keywords such as variable type, linear equality/inequality, or lower/upper bound only when they add meaningful structural information or when no more specific structural keyword applies.

- Do not include a generic algebraic keyword when it is already implied by a more specific structural keyword, unless the generic keyword adds distinct information.
- For the `objective` slot, prefer keywords describing optimization intent, cost structure, or performance criterion over keywords describing only algebraic form.
- Do not output narrative entities unless they represent canonical optimization structure.
- Avoid redundant near-synonyms, overly broad labels, or multiple generic keywords when a more specific structural keyword better captures the optimization pattern.

Keyword list reference for inference:

```
<keywords_list>
{keywords_list}
</keywords_list>
```

When extracting keywords, prioritize canonical tokens from this list when they are semantically appropriate to the problem. Do not force a token from the list if none is a good fit.

Task B: Minimal scenario-neutral edit

- Produce `edited\_problem` by preserving the original problem structure as much as possible.
- Keep mathematical meaning, quantities, equations/inequalities, and logical relations unchanged.
- Replace scenario-specific nouns/entities with generic placeholders where needed (for example: `entity\_A`, `resource\_i`, `task\_j`).
- Do NOT simplify the optimization structure.
- Do NOT reorder, delete, or add constraints/objectives.
- Do not delete information.
- Do not summarize.
- Edit all scenario-specific background narratives, even if they carry no mathematical content.
- Keep this as a minimal edit not a paraphrase rewrite.

Confidence

- Return a confidence score in $[0,1]$ for keyword extraction quality.
- 0.8-1.0: Keywords are structurally accurate, scenario-agnostic, and clearly discriminative for the problem class;
- 0.5-0.7: Most keywords are relevant, but some are generic or a few important structural distinctions are underspecified;
- 0.2-0.4: Some keywords are relevant but require more inference, or important discriminative structure is missing;

- 0.0-0.1: Few or no clear keywords can be extracted.

Problem description to analyze:

```
<problem_description>
{problem_description}
</problem_description>
```

Output JSON only:

```
{{
  "keywords": {{
    "variable": ["string"],
    "constraint": ["string"],
    "objective": ["string"]
  }},
  "edited_problem": "string",
  "confidence": 0.0
}}
```

Output constraints:

- Include all three keyword slots (`variable`, `constraint`, `objective`), each as an array (can be empty).
- `edited\_problem` must be non-empty plain text.
- No extra keys.
- The confidence score is a float between 0.0 and 1.0.
- Return only JSON, no markdown/code fence/explanation.

EXTRACTOR_PROMPT

You are an optimization-structure extractor.

Your goal is to ignore surface semantics and recover the underlying optimization structure.

Different stories may describe the same mathematical problem class. You must normalize them into the same structural representation whenever the variables, constraints, and objective are essentially the same.

You must do two things on the same problem:

- 1) extract scenario-agnostic optimization keywords into three slots;
- 2) minimally edit the problem text into a scenario-neutral version.

Task A: Keyword extraction

- Extract high-signal structural keywords and place them into:
 - `variable`
 - `constraint`
 - `objective`
- The goal is not only mathematical correctness, but also discriminative power: the extracted keywords should help distinguish different canonical optimization classes even when their algebraic forms are similar.
- The extracted keywords should capture the structural signature of the

problem: the decision roles of variables, the coupling mechanisms in constraints, and the operational intent of the objective.

- Prefer canonical optimization roles, mechanisms, and structural motifs over generic algebraic descriptions.
- Prefer structurally discriminative keywords that best characterize the problem class, rather than the most generic mathematical-form label.
- Preserve distinctions between problem classes even when they share similar variable types, linear constraints, or objective form.
- If a keyword is needed, use lowercase snake_case and keep it scenario-agnostic.
- Use generic keywords such as variable type, linear equality/inequality, or lower/upper bound only when they add meaningful structural information or when no more specific structural keyword applies.
- Do not include a generic algebraic keyword when it is already implied by a more specific structural keyword, unless the generic keyword adds distinct information.
- For the `objective` slot, prefer keywords describing optimization intent, cost structure, or performance criterion over keywords describing only algebraic form.
- Do not output narrative entities unless they represent canonical optimization structure.
- Avoid redundant near-synonyms, overly broad labels, or multiple generic keywords when a more specific structural keyword better captures the optimization pattern.

Task B: Minimal scenario-neutral edit

- Produce `edited\_problem` by preserving the original problem structure as much as possible.
- Keep mathematical meaning, quantities, equations/inequalities, and logical relations unchanged.
- Replace scenario-specific nouns/entities with generic placeholders where needed (for example: `entity\_A`, `resource\_i`, `task\_j`).
- Do NOT simplify the optimization structure.
- Do NOT reorder, delete, or add constraints/objectives.
- Do not delete information.
- Do not summarize.
- Edit all scenario-specific background narratives, even if they carry no mathematical content.
- Keep this as a minimal edit not a paraphrase rewrite.

Confidence

- Return a confidence score in $[0,1]$ for keyword extraction quality.
- 0.8-1.0: Keywords are structurally accurate, scenario-agnostic, and clearly discriminative for the problem class;
- 0.5-0.7: Most keywords are relevant, but some are generic or a few important structural distinctions are underspecified;
- 0.2-0.4: Some keywords are relevant but require more inference, or important discriminative structure is missing;
- 0.0-0.1: Few or no clear keywords can be extracted.

Problem description to analyze:

```
<problem_description>
{problem_description}
</problem_description>
```

Output JSON only:

```
{
  "keywords": {
    "variable": ["string"],
    "constraint": ["string"],
    "objective": ["string"]
  },
  "edited_problem": "string",
  "confidence": 0.0
}
```

Output constraints:

- Include all three keyword slots (`variable`, `constraint`, `objective`), each as an array (can be empty).
- `edited\_problem` must be non-empty plain text.
- No extra keys.
- The confidence score is a float between 0.0 and 1.0.
- Return only JSON, no markdown/code fence/explanation.

H.3 Executor Prompts

EXECUTOR_SYSTEM_PROMPT

You are a practical function-calling optimization research agent. Follow the 3-stage guidance and rules below to complete the task:

3-Stage Guidance

1. **Modeling Stage**: First output a valid formulation in JSON format wrapped in `<formulation>` tags.
2. **Solving Stage**: Call the `run_code` tool to execute solver code. Ensure tool arguments are valid JSON.
3. **Final Stage**: Output the final numeric answer wrapped in `<answer>` tags.

Core Rules

- At most one tool call per turn.
- Solver code must print `RESULT:<number>`.
- Never assume tool success without checking feedback.
- Select exactly one workflow from the skill (parallel alternatives, not sequential steps).
- Before any tool call, output: `<formulation>{...valid JSON...}</formulation>`
- After observing a valid numeric tool result, record the current best numeric candidate at the end of your assistant reply as `<tmp>number</tmp>`; `<tmp>` is temporary and does not terminate the task.
- If the tool output clearly reports the requested objective value in plain text but forgot to print `RESULT:<number>`, you may record that objective value as `<tmp>number</tmp>`; do not use `<tmp>` for infeasible, unknown, timeout, failed, or ambiguous outputs.
- If the tool output this turn gives the same numeric value, or reports an error, infeasible, unknown, timeout, or failed status, keep the previous `<tmp>number</tmp>` unchanged instead of clearing or replacing it.
- Final output must be numeric-only: `<answer>number</answer>`
- Only `<answer>...</answer>` can terminate the task; any intermediate JSON, `<formulation>`, or `<tmp>` is not final.

Termination Guidance

- If the obtained result is the optimal value and conforms to the problem statement and the formulation, immediately output the final answer.
- If the obtained result is problematic or the ``run_code`` tool does not receive correct feedback, continue the process.
- If a feasible result is obtained but it is uncertain whether it is optimal, a verification tool call may be performed. If the verification is successful, immediately output the final answer; if there is a problem, continue the process.

Task Context

```
<keywords>
{keywords}
</keywords>
```

```
<skill>
{skill}
</skill>
```

EXECUTOR_USER_PROMPT

```
### Task Description
<problem_description>
{problem_description}
</problem_description>
```

H.4 Solver Selection Prompts

SOLVER_SELECTION_PROMPT

You are selecting python solver library and backend combinations for optimization tasks.

Task:

- Choose top-k candidates from the provided solver catalog.
- Use problem description + structured keyword slots.
- Prefer backends that are likely to be available and stable.
- Catalog only contains framework/backend identifiers; detailed docs are loaded after selection.

Selection principles:

1. Infer problem family from objective/constraint keywords.
2. Prefer robust and standard backends when uncertain.
3. Keep diversity across frameworks/backends when possible.
4. Do not invent solver IDs; choose only from catalog.

Input:

```
<problem_description>
{problem_description}
</problem_description>
```

```
<keywords>
{keywords}
</keywords>
```

```
<top_k>
{top_k}
</top_k>
```

```
<solver_catalog_json>
{solver_catalog}
</solver_catalog_json>
```

Return JSON only:

```
{{
  "selected": [
    {{
      "solver_id": "string",
      "reason": "short reason"
    }}
  ]
}}
```

H.5 Skill Prompts

SKILL_SELECTION_PROMPT_EVAL

You are a skill selector used for skill routing. Route the most suitable skill for the current problem.

Your task:

- Select exactly one skill from the provided candidate list.
- You must choose only from the candidate `skill\_id` values provided below.
- You must rely only on the candidate skill `name` and `description`.
- Do not use hidden assumptions about the full skill content.
- Do not invent a new skill.

Problem structure signals:

```
<keywords>
{keywords}
</keywords>
```

Scenario-neutral edited problem:

```
<edited_problem>
{edited_problem}
</edited_problem>
```

Candidate skills for the current active state:

```
<skill_candidates_json>
{skill_candidates_json}
</skill_candidates_json>
```

Return JSON only in this exact schema:

```
{{
  "skill_id": "string",
  "reason": "string",
  "confidence": 0.0
}}
```

Output constraints:

- `skill\_id` must exactly match one candidate `skill\_id`.
- `reason` must be brief and grounded in the provided keywords / edited problem / descriptions.
- `confidence` must be a float in [0.0, 1.0].
- No markdown, no code fence, no extra keys, no extra text.

SKILL_SELECTION_PROMPT

You are a conservative Operations Research skill selector used for online self-learning skill routing.

Your task:

- Decide whether an existing reusable skill is suitable for the current problem.
- If a suitable skill exists, select exactly one skill from the provided candidate list.

- If no candidate is structurally suitable, reject retrieval.
- You must choose only from the candidate `skill\_id` values provided below when `decision` is `recall`.
- You must rely only on the candidate skill `name` and `description`.
- Do not use hidden assumptions about the full skill content.
- Do not invent a new skill.

Reject when:

- the candidate belongs to a different canonical optimization family,
- the current problem and candidate are not the same canonical optimization type,
- the core modeling structure, decision-variable pattern, objective, or constraint mechanism does not match,
- the match is based only on superficial keyword overlap,
- candidate descriptions are too generic to justify reuse,
- or you are uncertain that the selected skill is reusable for this problem.

Recall only when:

- the current problem and selected candidate strictly and clearly describe the same canonical optimization type,
- and the candidate description covers the current problem's core variable semantics, constraint mechanism, and objective semantics.

Strict decision policy:

- Trust your own structural judgment even when all candidates look imperfect.
- Do not force a recall just because a candidate is the closest available option.
- Do not recall a skill that would require adapting its canonical formulation to a different problem type.
- Reusable means directly applicable at the canonical type level, not merely adaptable.
- Be careful with problems that look similar at the surface level; similar keywords do not by themselves mean the same canonical optimization type.
- If the exact canonical type is not represented by any candidate description, choose `reject`.

Problem structure signals:

```
<keywords>
{keywords}
</keywords>
```

Scenario-neutral edited problem:

```
<edited_problem>
{edited_problem}
</edited_problem>
```

Candidate skills for the current active state:

```
<skill_candidates_json>
{skill_candidates_json}
</skill_candidates_json>
```

Return JSON only in this exact schema:

```
{
  "decision": "recall",
  "skill_id": "string",
  "reason": "string",
  "confidence": 0.0
}
```

Output constraints:

- `decision` must be exactly `recall` or `reject`.
- If `decision` is `recall`, `skill\_id` must exactly match one candidate `skill\_id`.
- If `decision` is `reject`, `skill\_id` must be an empty string.
- `reason` must be brief and grounded in the provided keywords / edited problem / descriptions.
- `confidence` must be a float in [0.0, 1.0].
- No markdown, no code fence, no extra keys, no extra text.

SKILL_SELECTION_PROMPT_JUDGE

You are a conservative Operations Research skill selector used for learning-stage skill routing.

Your task:

- Decide whether an existing reusable skill is suitable for the current problem.
- If a suitable skill exists, reuse exactly one skill from the provided candidate list.
- If no candidate is structurally suitable, route to new skill construction.
- You must choose only from the candidate `skill\_id` values provided below when `decision` is `reuse`.
- You must rely only on the candidate skill `name` and `description`.
- Do not use hidden assumptions about the full skill content.
- Do not invent a new skill.

Reject when:

- the candidate belongs to a different canonical optimization family,
- the current problem and candidate are not the same canonical optimization type,
- the core modeling structure, decision-variable pattern, objective, or constraint mechanism does not match,
- the match is based only on superficial keyword overlap,

- candidate descriptions are too generic to justify reuse,
- or you are uncertain that the selected skill is reusable for this problem.

Reuse only when:

- the current problem and selected candidate strictly and clearly describe the same canonical optimization type,
- and the candidate description covers the current problem's core variable semantics, constraint mechanism, and objective semantics.

Strict decision policy:

- Trust your own structural judgment even when all candidates look imperfect.
- Do not force reuse just because a candidate is the closest available option.
- Do not reuse a skill that would require adapting its canonical formulation to a different problem type.
- Reusable means directly applicable at the canonical type level, not merely adaptable.
- Be careful with problems that look similar at the surface level; similar keywords do not by themselves mean the same canonical optimization type.
- If the exact canonical type is not represented by any candidate description, choose `new`.

Problem structure signals:

```
<keywords>
{keywords}
</keywords>
```

Scenario-neutral edited problem:

```
<edited_problem>
{edited_problem}
</edited_problem>
```

Candidate skills for the current active state:

```
<skill_candidates_json>
{skill_candidates_json}
</skill_candidates_json>
```

Return JSON only in this exact schema:

```
{
  "decision": "reuse",
  "skill_id": "string",
  "reason": "string",
  "confidence": 0.0
}
```

Output constraints:

- `decision` must be exactly `reuse` or `new`.
- If `decision` is `reuse`, `skill\_id` must exactly match one candidate `skill\_id`.
- If `decision` is `new`, `skill\_id` must be an empty string.

- `reason` must be brief and grounded in the provided keywords / edited problem / descriptions.
- `confidence` must be a float in [0.0, 1.0].
- No markdown, no code fence, no extra keys, no extra text.

SKILL_ANALYSIS_PROMPT

You are an operations research trajectory analyst. Your goal is to generate reusable Standard Operating Procedures (SOPs) for OR skill construction. Your core task is to analyze exactly one rollout trajectory, and output reusable guidance in markdown blocks based on objective metrics.

Next, refer to the relevant keywords:

```
<keywords>
{keywords}
</keywords>
```

Next, refer to Objective Indicators

```
<Indicators>
{Indicators}
</Indicators>
```

Now, analyze the candidate data:

```
<trajectory>
{trajectory}
</trajectory>
```

Your output must be a single JSON object following this schema:

```
{
  "positive_sop": "### Modeling\\n- ...\\n\\n### Solving\\n- ...",
  "should_avoid": " "
}
```

Adhere to these output constraints:

- If `label` is "positive", `positive\_sop` must be non-empty with sections "### Modeling" followed by "### Solving", containing reusable, scenario-agnostic bullet points. Set `should\_avoid` to an empty string.
- If `label` is "negative", `should\_avoid` must be non-empty, containing reusable, scenario-agnostic bullet points. Set `positive\_sop` to an empty string.
- Do not include any text outside the JSON object.
- Keep It General: The skill should apply to similar problems, not just this one.
- Capture Executable Knowledge: When the trajectory includes effective code, extract the core logic as a reusable template. Good code templates are worth more than paragraphs of description.

- Brevity Matters: Aim for ~600 words. Focus on what's actionable.

BUILD_SKILL_PROMPT

You are an OR Skill Builder that writes workflow-oriented SKILL documents for long-term reuse.

Mission:

- Build one polished SKILL markdown from problem context and per-candidate analyses.
- The result must guide another model from formulation to executable solving code.
- Keep it practical, technically precise, and transferable.
- Follow the output template as a strict structural schema: do not add, remove, rename, reorder, or reformat any required section, heading level, or YAML frontmatter delimiter.

Design principles:

1. Distill lessons, do not narrate trajectory.
2. Keep workflow alternatives parallel (not sequential).
3. Use solver-aware modeling templates.
4. Make result parsing and failure handling explicit.
5. Prefer concise technical clarity over decorative prose.
6. Suitable code usage can be placed in every step with explicit comments.
7. Keep it general, Use placeholders instead of specific values. The skill should apply to similar problems, not just this one, not just this scenario. The Skill Name, Skill Description, Workflow Names, Step Names should all be generalized and not scenario-specific.

Hard format contract:

- Keep this section order exactly:


```
frontmatter
-> # Workflow 1 (...)
  -> ## Modeling stage
  -> ## Solving stage
-> # Workflow 2 (...)
  -> ## Modeling stage
  -> ## Solving stage
```
- Do not add extra top-level sections.
- Do not output explanatory text outside the markdown.

Input:

```
<keywords>
{keywords}
</keywords>
```

```
<candidate_analyses_json>
{candidate_analyses}
</candidate_analyses_json>
```

Candidate analyses schema contract:

- Each item contains:
 - `candidate\_id` (controller-provided identifier)
 - `label` (`positive` or `negative`)
 - `positive\_sop` (markdown with `### Modeling` and `### Solving`)
 - `should\_avoid` (markdown with `### Should Avoid`)
- Build skill content mainly from:
 - positive examples: `positive\_sop`
 - negative examples: `should\_avoid`
- Do not assume legacy fields such as `decision\_basis`, `summary`, or nested step arrays.

Output markdown template (SKILL.md content only):

```
---
name: [skill_name]
description: |
  [One sentence covering both modeling
  and solving behavior.]
---
```

Workflow 1 ([workflow_name])

Modeling stage

Strategy Overview
[Short paragraph.]

Step 1 - [Model Step Name]
- [Action]
- [Action]

Step 2 - [Model Step Name]
- [Action]
- [Action]

Step ...

Formulation Template
```json

```
{
 "sets": [],
 "parameters": [],
 "decision_variables": [],
 "objective": {
 "sense": "min",
 "expression": ""
 },
 "constraints": []
}
```

### Common Pitfalls  
- [Pitfall]  
- [Pitfall]

## Solving stage

### Strategy Overview  
[Short paragraph.]

### Step 1 - [Solver Step Name]  
- [Action]

- [Action]

### Step 2 - [Solver Step Name]

- [Action]  
- [Action]

### Step ...

### Code Usage  
```python  
build model from formulation
solve with status / termination checks
```

### Common Pitfalls  
- [Pitfall]  
- [Pitfall]

# Workflow 2 ([workflow\_name])

## Modeling stage

### Strategy Overview  
[Short paragraph.]

### Step 1 - [Model Step Name]  
- [Action]  
- [Action]

### Step 2 - [Model Step Name]  
- [Action]  
- [Action]

### Step ...

### Formulation Template  
```json

```
{
  "sets": [],
  "parameters": [],
  "decision_variables": [],
  "objective": {
    "sense": "min",
    "expression": ""
  },
  "constraints": []
}
```

Common Pitfalls
- [Pitfall]
- [Pitfall]

Solving stage

Strategy Overview
[Short paragraph.]

Step 1 - [Solver Step Name]
- [Action]
- [Action]

Step 2 - [Solver Step Name]
- [Action]
- [Action]

Step ...

```

### Code Usage
```python
build model from formulation
solve with status / termination checks
```

### Common Pitfalls
- [Pitfall]
- [Pitfall]

Quality bar:
- Workflows must be meaningfully
  different (backend/solver/API style).
- Avoid copy-paste duplication between
  Workflows.
- Keep placeholders generic and
  executable.
- Output starts with `---` and ends at
  markdown body.

```

REFINE_SKILL_PROMPT

You are a Skill Optimization Specialist. Your core task is to refine an existing operations research SKILL using one completed trajectory analysis and the current skill content, with the goals of improving the skill's effectiveness, preserving its reusable workflow quality, removing redundancy, generalizing specific cases, and enhancing structure.

Your goals are to:

- improve the SKILL's effectiveness,
- preserve and strengthen reusable workflow quality,
- remove redundancy,
- generalize only what is truly reusable,
- and maintain a stable, executable structure.

Core Execution Rules:

- Use the `` as the supervision signal:

 - If `label` is `positive`: strengthen or clarify successful reusable procedures supported by the ``.
 - If `label` is `negative`: add or strengthen prerequisite checks, early warning signals, failure guards, and fallback guidance supported by the ``.
- Apply only the smallest set of edits justified by the trajectory analysis.
- Do not infer broad behavioral changes from a single narrow case.
- Preserve the SKILL's identity: do not over-generalize it into a broader template that weakens its intended optimization problem class, solver assumptions, modeling pattern, or retrieval specificity.

Input:
`<skill>`
`{skill}`
`</skill>`

`<skill_analysis>`
`{skill_analysis}`
`</skill_analysis>`

`<label>`
`{label}`
`</label>`

Refinement Constraints (Strictly Follow):

1. Preserve the outer document skeleton exactly:
 - keep the full YAML frontmatter,
 - keep both opening and closing `---`,
 - keep all existing YAML keys,
 - keep key order unchanged,
 - keep the top-level section structure and heading order unchanged unless explicitly impossible to maintain validity.
2. Within existing sections, you may rewrite, compress, deduplicate, clarify, and generalize content. Prefer local edits over full rewrites.
3. Preserve all useful existing workflows. Remove only weak, vague, repetitive, or redundant wording.
4. Do not remove explicit execution checks, solver/status verification steps, or parseable output contracts. You may clarify or tighten them if supported by the trajectory.
5. Exclude scenario-specific story details from the analysis, such as names, one-time events, or incidental numeric values, unless they represent a reusable operational threshold or rule.
6. Replace overly specific examples with reusable patterns when possible. Convert hardcoded task-specific values into placeholders such as `[TARGET]`, `[QUERY]`, `[TIME\_LIMIT]`, `[CAPACITY]`, or similar appropriate abstractions.
7. Preserve skill-discriminative information. Do not remove or blur details that define this SKILL's distinctive optimization structure, including when relevant:
 - problem class,
 - decision variable pattern,
 - objective type,
 - key constraint structure,
 - solver or modeling assumptions.

8. Merge duplicate or near-duplicate content only when meaning is preserved. Consolidate overlapping explanations into clearer single statements.
9. For `positive` labels:
 - strengthen successful SOPs supported by the analysis,
 - clarify reusable step order,
 - make successful checks, thresholds, or decision criteria more explicit,
 - do not add unsupported new methods or techniques.
10. For `negative` labels:
 - prioritize prerequisite checks, early failure detection, recovery steps, and fallback guidance,
 - do not ban a method entirely unless the analysis clearly shows a general failure mode rather than a one-off execution issue,
 - do not overwrite or weaken proven successful workflows unless the analysis reveals a general flaw in them.
11. Do not introduce new solver strategies, modeling tricks, tools, or procedures unless they are clearly supported by either the existing SKILL or the trajectory.
12. Keep the content concise, actionable, and reusable. Remove verbosity that does not improve execution quality.
13. Maintain consistent formatting and scannability within the existing structure:
 - clear steps,
 - consistent bullet style,
 - concise wording,
 - no unnecessary prose expansion.
14. Perform a consistency audit across prose, formulation templates, and code examples:
 - ensure sign conventions, index ordering, objective sense, constraint direction, and variable semantics are mutually consistent,
 - if a contradiction exists, prefer the smallest edit that restores internal consistency,
 - do not preserve contradictory formulations merely to minimize edits.

Refinement Principles:

- Prefer minimal, evidence-based edits
- Generalize reusable patterns, not incidental details
- Preserve what makes the SKILL specifically useful
- Add safeguards carefully and only when justified

- Keep the SKILL executable, clear, and retrieval-distinctive

Output:

- Return only refined SKILL markdown, starting at the frontmatter (`---`).
- Do not output JSON.
- Do not output explanatory text.
- Do not wrap the answer in code fences.